

System Programming Project 3

담당 교수 : 김영재 교수님

이름 : 김동건

학번 : 20211507

1. 개발 목표

- 해당 프로젝트에서 구현할 내용을 간략히 서술.
- (주식 서버를 만드는 전체적인 개요에 대해서 작성하면 됨.)

이번 프로젝트에서는 동시 주식 서버를 구현하였다. 한 명의 client를 넘어 여러 client들의 동시 접속 및 서비스를 위한 Concurrent stock server를 구현하였다. client를 통해 buy sell show exit 4개의 입력을 받아 주식을 사고 파는 시장을 구현하였다. 이때, 여러 client의 동시 접속과 서비스 수행을 위해 두 가지 방법을 구현하였다. 하나는 Event-driven Approach이고, 다른 하나는 Thread-based Approach이다. 마지막엔 두 방법의 성능을 비교하여 분석하였다.

2. 개발 범위 및 내용

A. 개발 범위

- 아래 항목을 구현했을 때의 결과를 간략히 서술

1. Task 1: Event-driven Approach

우선, 주식 관리를 위한 binary search tree를 구현하였다. 강의자료를 참고하여 pool을 관리할 수 있도록 구현하였다. listen fd를 통해 요청을 accept, select하여 ready 상태인 것을 connfd에 배정하였다.

2. Task 2: Thread-based Approach

Task 1에서 구현한 binary search tree와 파싱하는 부분은 그대로 이용하였고, pool을 통한 event driven 관련 코드를 thread based로 수정하였다. 각 thread가 client 하나를 관리하도록 하였고, 그 과정에서 동기화 문제를 해결하기 위해 mutex를 사용하였다.

3. Task 3: Performance Evaluation

Task 1과 Task 2의 성능 차이를 분석하였다. 요청의 종류와 사용자 수를 변화시켜가며 동시 처리율을 측정하였고, 수업 내용과 연관시켜 분석 결과를 정리하였다.

B. 개발 내용

- 아래 항목의 내용만 서술

- (기타 내용은 서술하지 않아도 됨. 코드 복사 붙여 넣기 금지)

- **Task1 (Event-driven Approach with select())**

✓ Multi-client 요청에 따른 I/O Multiplexing 설명

이번 프로젝트에서는 select으로 I/O Multiplexing을 구현하였다. listen fd를 받는 pool을 만든 뒤, select로 요청을 받는다. 만약 연결이 필요하다면 connfd로 pool에 추가한 뒤, check client로 pool에서 ready된 것과 통신하여 기능을 수행한다. 이를 통해 하나 이상의 I/O event를 제어한다.

✓ epoll과의 차이점 서술

select에서는 fd를 순회하며 ready인 것을 수행하였다. epoll에서는 fd의 상태 변화를 커널이 관리한다는 차이가 있다. 즉, descriptor가 많은 경우에도 안정적인 성능을 낸다.

- **Task2 (Thread-based Approach with pthread)**

✓ Master Thread의 Connection 관리

Master-Worker방식을 사용하여 thread_based approach를 구현하였다. Master가 연결을 감지하고 accept하고 버퍼에 descriptor를 추가한다. worker 스레드에서는 버퍼에 descriptor 존재여부를 감지하여 그것을 삭제하고 연결한다. 각 client마다 요청을 처리하게 된다.

✓ Worker Thread Pool 관리하는 부분에 대해 서술

main실행 후, Pthread_create로 미리 스레드들을 생성한다. 각 thread에서는 master가 sbuf에 삽입해둔 것을 탐지하여 sbuf_remove를 통해 삭제하고 echo_cnt에서 명령을 수행한다.

- **Task3 (Performance Evaluation)**

✓ 얻고자 하는 metric 정의, 그렇게 정한 이유, 측정 방법 서술

수행 시간과 동시 처리율을 얻고자 하였다. 우선, 정확한 동작을 하는 프로그램이 있을 때, 시간은 프로그램 성능의 가장 중요한 역할을 하는 metric이라 수행 시간을 측정하였다. multiclien 파일에서 명세서에서 알려진 gettimeofday()로 main 전 후 시간을 측정하였다. ms단위로 계산하였고, 명세서 링크에 있는 코드의 수식을 그대로 사용하였다.

```
e_usec = ((end.tv_sec * 1000000) + end.tv_usec) - ((start.tv_sec * 1000000)
+ start.tv_usec);
```

동시 처리율은 앞서 말한 시간측정을 이용하였고,(요청 수) / (수행 시간)으로 정의하였다. 이때, 요청수는 num_client * ORDER_PER_CLIENT이다.

해당 값이 너무 작기 때문에 10^8 을 곱하여 살펴보았다.

✓ Configuration 변화에 따른 예상 결과 서술

수업에서 배운 것을 토대로 예상했을 때는 thread based와 event driven이 비슷한 경향성을 띌 것이라 생각한다. event driven의 경우 루프를 순회하며 매번 확인을 해야하기 때문에, 해당 작업에서 n이 커질수록 오버헤드가 커질 거라 생각하였다. 반면 thread-based의 경우 n이 커질수록 context switch 비용이 증가하여 마찬가지로 오버헤드가 커질 거라 생각하였다. 다만, 두 개 중 어느 영향이 더 클지는 예측할 수 없다고 판단하여 실제 실험을 통해 확인하여 분석하였다.

C. 개발 방법

- B.의 개발 내용을 구현하기 위해 어느 소스코드에 어떤 요소를 추가 또는 수정할 것인지 설명. (함수, 구조체 등의 구현이나 수정을 서술)

우선, 1과 2에서 모두 binary search tree를 사용하여 주식을 관리하였다.

```
typedef struct stock {
    int id, cnt, price, idx;
    struct stock *left, *right;
} stock;
stock* root;

int client_count = 0;
int stock_cnt = 0; // 주식 개수
int idx[MAX_STOCK_NUM]; // stock.txt에 저장되어 있는 순서..
// idx[0]가 3이라는 의미: stock.txt 0번째 주식 id가 3이라는 의미..
// 입력된 순서대로 출력하기 위해..
```

다만, 출력 시에도 입력과 같은 순서를 유지해야 하기 때문에, 실제 로드했을 때 인덱스를 관리할 배열을 따로 만들어 관리하였다.

```
stock* insert_stock(stock* u, stock* item) { // item을 삽입
    if (!u) return item;
    if (u->id == item->id) u->cnt += item->cnt; // 이미 있는 경우
    else if (u->id > item->id) u->left = insert_stock(u->left, item);
    else u->right = insert_stock(u->right, item);
    return u;
}

int find_stock_by_id(stock* u, int id, char* buf) { // 트리에 id가 무조건 있다고 가정
    if (u->id == id) return sprintf(buf, "%d %d %d\n", u->id, u->cnt, u->price);
    else if (u->id > id) return find_stock_by_id(u->left, id, buf);
    else return find_stock_by_id(u->right, id, buf);
}

void show_stock(char* buf) { // buf에 쓰기, return 값은 쓴 길이
    // 입력된 순서대로 출력하기 위해..
    for (int i = 0; i < stock_cnt; i++) buf += find_stock_by_id(root, idx[i], buf);
    return;
}
```

binary search tree를 구현하였고, find_stock_by_id는 id가 주어졌을 때 해당 노드를 찾는 함수이다.

show_stock에서는 입력한 순서대로 해당 작업을 진행하였다.

```
void load_stock() {
    FILE* fp = fopen("stock.txt", "r");
    int id, cnt, price;
    while (fscanf(fp, "%d %d %d", &id, &cnt, &price) != EOF) {
        stock* u = (stock*)malloc(sizeof(stock));
        u->id = id, u->cnt = cnt, u->price = price, u->idx = stock_cnt;
        idx[stock_cnt++] = id;
        u->left = u->right = NULL;
        root = insert_stock(root, u);
    }
    fclose(fp);
    return;
}

void save_stock() {
    FILE* fp = fopen("stock.txt", "w");

    // buf에 저장 후, stock.txt에 write
    char buf[MAXLINE] = {'\0'};
    show_stock(buf), fprintf(fp, "%s", buf);
    fclose(fp);
    return;
}

// stock.txt는 항상 사고 팔 주식에 있는 것으로 가정하고 과제를 진행하시면 됩니다.
int update_stock(stock* u, int id, int cnt) {
    if (u->id == id) {
        if (u->cnt + cnt < 0) return 0;
        u->cnt += cnt;
        return 1;
    }
    else if (u->id > id) return update_stock(u->left, id, cnt);
    else return update_stock(u->right, id, cnt);
}
```

load_stock은 stock.txt에서 주식 정보를 입력받아 트리에 저장하는 함수이다.

save_stock은 stock.txt 파일에 주식 정보를 저장하는 함수이다.

update_stock은 사고 팔 때 업데이트를 구현하였고, buy의 경우 업데이트가 불가능한 경우 0, 나머지 경우에는 1을 리턴하였다.

또, 1, 2 모두 sigint handler를 구현하여 ctrl + C로 종료될 때, save_stock을 하도록 하였다.

이제, task1 event driven 부분이다.

강의 자료의 pool 코드를 사용하였다.

```
// pool code: 강의자료/교과서 코드 사용
typedef struct {
    int maxfd; // 가장 큰 fd 번호
    fd_set read_set, ready_set;
    int nready; // ready set에서 1 개수
    int maxi;
    int clientfd[FD_SETSIZE];
    rio_t clientrio[FD_SETSIZE]; // set of read buffers
} pool;
```

init_pool로 pool을 아무 연결 없는 상태로 초기화 한 뒤, listen fd만 1로 설정한다.
 add_client에서는 빈 슬롯에 추가하고, readyset에 connfd를 추가한다.
 check_clients에서는 ready 상태인 것을 한 줄 씩 처리한다. 이는 루프를 돌면서 실행
 하고, 입력을 받은 뒤, show buy sell exit을 확인 후 해당 작업을 수행 후 client에게
 출력까지 진행한다.

또, 연결된 client_count를 관리하며, 해당 client가 0이 될 때마다 save_stock을 진행
 하였다.

다음으로, task2 thread based 부분이다.

여기서도 강의자료의 sbuf를 사용하였다.

```
// sbuf (강의자료 참고)
typedef struct {
    int* buf; //buffer array
    int n; // 최대 슬롯
    int front; // buf[(front + 1) % n]: first
    int rear; // buf[rear % n] : last
    sem_t mutex; // buf 접근 protect
    sem_t slots; // 슬롯
    sem_t items; // 아이템
} sbuf_t;
sbuf_t sbuf;
```

semaphore를 통해 master와 worker thread간의 동기화 문제를 해결하였다. mutex를
 통해 buf 접근을 제한하여 동기화 문제를 해결하였다.

sbuf_init에서는 구조체를 초기화 한다.

sbuf_deinit에서는 구조체를 free한다.

sbuf_insert에서는 사용자로부터 connect 요청이 왔을 때, 세마포어를 통해 buffer를
 안전한 상태로 만든 뒤, 아이템을 추가하고 다시 lock을 해제한다.

sbuf_remove에서는 첫 원소를 삭제하고 그 원소를 리턴하게 구현하였고, insert에서
 와 마찬가지로 lock을 활용하여 안전하게 관리하여 주었다.

insert/remove 모두 다른 스레드 접근을 막아야 하기 때문에 lock을 사용하였다.

task2에서는 client count를 관리 할 용으로 mutex를 선언하여 증가와 감소 전후로

lock/unlock을 통해 스레드끼리 비정상적으로 수정하는 경우가 없게 안전하게 관리하였다.

task2 에서 파싱하는 부분은 task1을 그대로 사용하였다.

마지막으로 task3의 경우 명세서에서 언급한 sys/time.h의 gettimeofday를 이용하였다.

multiclient.c의 main 전후로 시간을 측정하고, 끝에서 이를 출력하여 task3 분석에 활용하였다.

```
// 여기 아래는 시간 측정용 코드
// printf 주석 후 제출하기..
gettimeofday(&end, 0);

e_usec = ((end.tv_sec * 1000000) + end.tv_usec) - ((start.tv_sec * 1000000) + start.tv_usec);

printf("elapsed time: %lu microseconds\n", e_usec);
printf("clients: %d\n", num_client);
printf("total order: %d\n", num_client * ORDER_PER_CLIENT);
printf("동시 처리율 %0.5lf\n", 1.0 * 1e8 * ORDER_PER_CLIENT * num_client / e_usec);
```

3. 구현 결과

- 2번의 구현 결과를 간략하게 작성
- 미처 구현하지 못한 부분에 대해선 디자인에 대한 내용도 추가

먼저 task1과 2에서 binary search tree를 통해 주식 정보를 관리할 수 있게 되었다. 또, 클라이언트가 0명이 되면 stock.txt를 업데이트 할 수 있게 구현하였고, 그 외에도 sigint로 종료될 때도 올바르게 저장할 수 있게 시그널 핸들러를 등록하였다.

- task1 의 경우 connected descriptor를 pool 구조체로 관리할 수 있게 되었다. select를 통해 요청이 있다면 ready set에 추가한 뒤, add_client를 진행한다. 그 후 루프를 돌 때마다 이를 순회하여 체크한 뒤 클라이언트와 상호작용이 가능하게 되었다.
- task2의 경우에는 master thread가 client 요청을 관리하고 worker 스레드들은 buf에서 connfd를 삭제 후 그것을 처리할 수 있게 되었다. task2의 경우는 client count를 관리할 때도 역시 mutex를 이용하여 안전하게 관리하였다.
- task3의 경우 2에서 서술한 코드를 통해 multiclient 실행 전후로 아래와 같은 출

력 메시지가 나오게 되었다.

```
[sell] success
1 44 1000
5 24 2000
3 33 3000
2 11 4000
4 29 5000
6 7 6000
7 3 7000
8 20 8000
10 6 9000
9 2 10000
[buy] success
elapsed time: 10007649 microseconds
clients: 4
total order: 40
동시 처리율 399.69427
cse20211507@cspro:~/CSE4100/prj3/task1$
```

4. 성능 평가 결과 (Task 3)

- 강의자료 슬라이드의 내용 참고하여 작성 (측정 시점, 출력 결과 값 캡처 포함)

- 클라이언트 개수에 따른 비교

order는 10으로 고정하였고, 각 task별로 동시 처리율을 측정하였다.

- task1

```
elapsed time: 10005803 microseconds
clients: 1
total order: 10
동시 처리율 99.94200
```


elapsed time: 10006667 microseconds
clients: 2
total order: 20
동시 처리율 199.86675

elapsed time: 10007228 microseconds
clients: 4
total order: 40
동시 처리율 399.71109

elapsed time: 10007335 microseconds
clients: 8
total order: 80
동시 처리율 799.41363

elapsed time: 10007973 microseconds
clients: 16
total order: 160
동시 처리율 1598.72534

elapsed time: 10008803 microseconds
clients: 32
total order: 320
동시 처리율 3197.18552

elapsed time: 10012125 microseconds
clients: 64
total order: 640
동시 처리율 6392.24940

elapsed time: 10019026 microseconds
clients: 128
total order: 1280
동시 처리율 12775.69297

elapsed time: 10037482 microseconds
clients: 256
total order: 2560
동시 처리율 25504.40439

elapsed time: 10630693 microseconds
clients: 512
total order: 5120
동시 처리율 48162.42930

- task2

```
elapsed time: 10005794 microseconds  
clients: 1  
total order: 10  
동시 처리율 99.94209
```

```
elapsed time: 10006984 microseconds  
clients: 2  
total order: 20  
동시 처리율 199.86042
```

```
elapsed time: 10007565 microseconds  
clients: 4  
total order: 40  
동시 처리율 399.69763
```

```
elapsed time: 10007523 microseconds  
clients: 8  
total order: 80  
동시 처리율 799.39861
```

```
elapsed time: 10008004 microseconds  
clients: 16  
total order: 160  
동시 처리율 1598.72038
```

```
elapsed time: 10009208 microseconds  
clients: 32  
total order: 320  
동시 처리율 3197.05615
```

```
elapsed time: 10084418 microseconds  
clients: 64  
total order: 640  
동시 처리율 6346.42475
```

```
elapsed time: 10016830 microseconds  
clients: 128  
total order: 1280  
동시 처리율 12778.49379
```

```

elapsed time: 20018940 microseconds
clients: 256
total order: 2560
동시 처리율 12787.88987

```

```

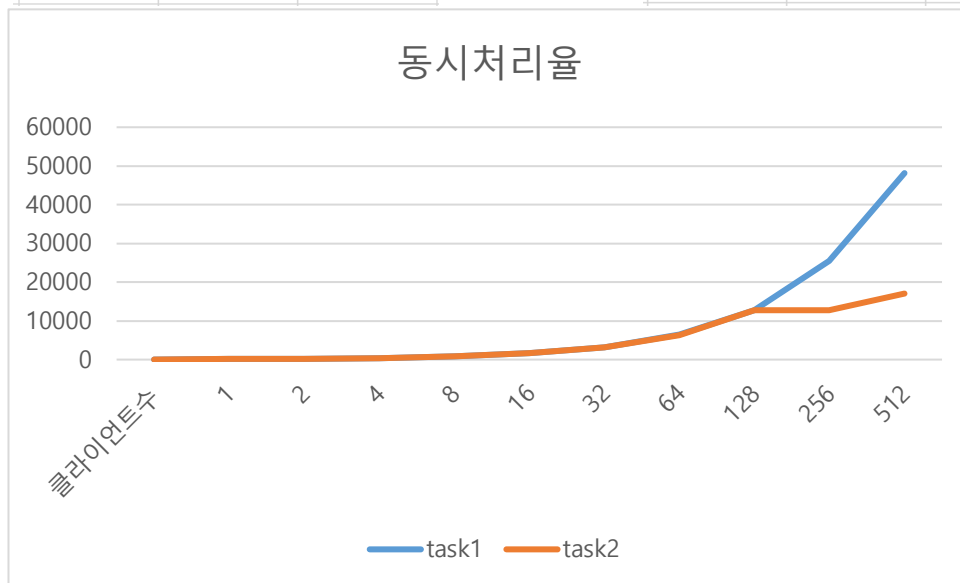
elapsed time: 30029293 microseconds
clients: 512
total order: 5120
동시 처리율 17050.01846

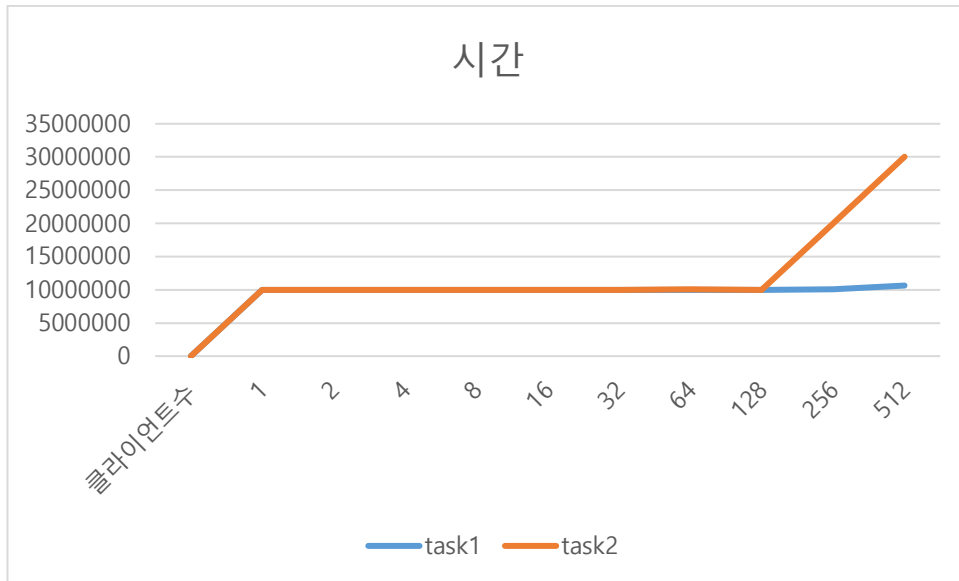
```

이를 표와 그래프로 정리해보면 아래와 같다.

	task1	task2
클라이언트수	동시처리율	동시처리율
1	99.942	99.94209
2	199.86675	199.86042
4	399.71109	399.69763
8	799.41363	799.39861
16	1598.72534	1598.72038
32	3197.18552	3197.05615
64	6392.2494	6346.42475
128	12775.693	12778.4938
256	25504.4044	12787.8899
512	48162.4293	17050.0185

	task1	task2
클라이언트수	시간	시간
1	10005803	10005794
2	10006667	10006984
4	10007228	10007565
8	10007335	10007523
16	10007973	10008004
32	10008803	10009208
64	10012125	10084418
128	10019026	10016830
256	10037482	20018940
512	10630693	30029293





별 차이가 없을 거라는 예상과 다르게 스레드 기반에서 256부터 클라이언트가 많아질수록 동시처리율 증가율이 급격하게 감소한다. 또, 시간 역시 급격하게 증가하는 모습을 보인다. event driven의 경우에는 동시처리율이 비교적 일정한 증가율을 보이고 있다.

이는 스레드 생성과 관리의 오버헤드때문이라고 생각할 수 있다. 또, 스레드가 많아질수록 스레드 간의 context switch 가 자주 일어날 것이고 이에 따른 비용으로 성능저하가 있는 것으로 보여진다. 스레드별로 스택을 관리하므로 메모리도 많이 사용할 것이다. 마지막으로 lock으로 인한 동기화 경합이 심해져 성능저하가 있을 것이다.

반면, event driven의 경우 의미없는 컨텍스트 스위칭이 없고, 스레드가 없기 때문에 메모리도 적고, lock을 사용하지 않아 이에 따른 손해도 없어서 비교적 좋은 성능을 유지하는 것으로 보인다. 또, 간단한 프로젝트라 각 이벤트마다 오래걸리는 작업이 없어서 event가 더 좋은 성능을 낸 것이라 생각한다.

- 워크로드에 따른 분석

- 다음으로는 워크로드에 따른 분석을 진행해 보았다. 다만, 모든 요청을 처리하는 경우는 앞선 분석에서 하였기 때문에 생략하였다. 또, buy와 sell의 경우 내부적으로 bst 탐색에서 같은 구조로 실행되므로 차이가 없을 것이라 예상하여 프로그램을 실행해 보았다. client 수는 512로 고정하였고, 사용자당 요청도 10으로 고정하

여 실험을 진행하였다. 다만, show와 sell, buy의 차이를 더 극대화하기 위해 주식의 개수를 기존 10개에서 20개로 변경하여 실험하였다. 첫 결과가 task1, 두 번째 결과가 task2이다.

사진은 하나만 첨부하였지만, 실제로는 5번씩 실험하였고, 평균값과 가장 가까운 결과를 첨부하였다.

multi_client if문을 수정해가며 실험하였다.

1. show만 요청하는 경우

```
elapsed time: 10097064 microseconds  
clients: 512  
total order: 5120  
동시 처리율 50707.80972
```

```
elapsed time: 30028384 microseconds  
clients: 512  
total order: 5120  
동시 처리율 17050.53459
```

2. buy만 요청하는 경우

```
Not enough left stocks  
elapsed time: 10079328 microseconds  
clients: 512  
total order: 5120  
동시 처리율 50797.03726
```

```
elapsed time: 30025708 microseconds  
clients: 512  
total order: 5120  
동시 처리율 17052.05419
```

3. sell만 요청하는 경우

```
elapsed time: 10091942 microseconds  
clients: 512  
total order: 5120  
동시 처리율 50733.54563
```

```
elapsed time: 30024010 microseconds  
clients: 512  
total order: 5120  
동시 처리율 17053.01857
```

4. buy와 sell을 섞어서 요청하는 경우

```
elapsed time: 10089430 microseconds  
clients: 512  
total order: 5120  
동시 처리율 50746.17694
```

```
elapsed time: 30023779 microseconds  
clients: 512  
total order: 5120  
동시 처리율 17053.14977
```

task1, 2 에서 모두 show의 경우 bst를 관리하긴 하지만, 결국 입력 순서를 보장하여 출력하여야 하기 때문에 $O(N^2)$ 에 출력하게 작성하였고, 실제로 bst에서 id를 찾아 업데이트하는 buy sell보다 조금 더 오래걸리는 것을 확인하였다. 다만, 그 차이가 생각만큼 드라마틱하게 크지는 않았다.

마지막으로, 입출력시 걸리는 시간에 대해 분석해 보았다. 서버의 성능을 측정해야 하기 때문에, multiclient에서는 출력을 모두 없애고 실험하였다. 다만, 서버 -> 사용자로 보내는 명령의 경우 실제로도 주식 서버에서 해야하는 일이기 때문에 그대로 유지한 채로 실험하였다.

정리하면 이번 프로젝트에서, event - thread간의 차이가 생각보다 크게 나타났다. thread의 경우 사용자 수가 많아질수록 좋지 않은 성능을 보였고, 이에 대한 분석과 이유는 위에서 서술하였다. 다만, 작업 간의 성능 차이는 생각보다 크게 나타나지 않았다. 특히, 트리를 redblack tree같은 밸런스드 트리를 사용하지 않았는데도 말이다. 다만, 이는 n 이 훨씬 커지면 더 큰 차이를 나타낼 것이라 생각한다.